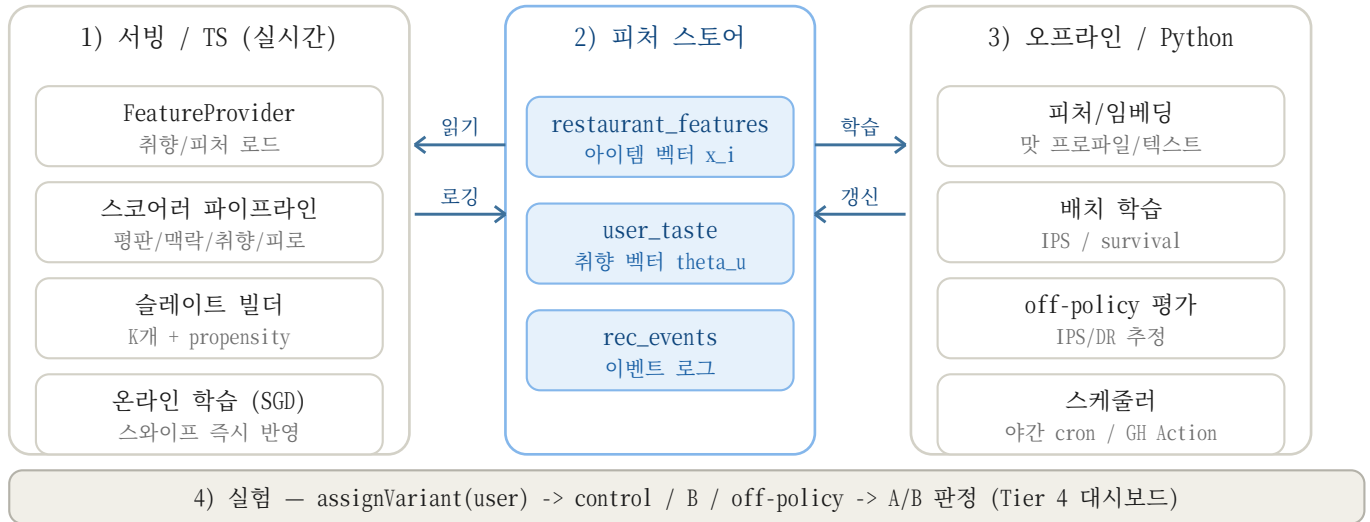


런치 엔진 아키텍처 설계

온라인 서빙 / 피처 스토어 / 오프라인 학습 — 2026-06-23

핵심 결정: 온라인(서빙)과 오프라인(학습)을 분리하고, 그 사이를 '피처 스토어'가 잇는 표준 추천 아키텍처. 지금 스택(Node/Express + Postgres)을 거의 그대로 쓰고, 무거운 ML만 Python으로 분리한다. 두 언어는 직접 엮이지 않고 DB(피처 스토어)로만 소통한다.



[그림 1] 서빙(TS)이 이벤트를 로깅 -> 오프라인(Python)이 그 로그로 학습 -> 피처 스토어에 갱신 -> 서빙이 다시 읽는 피드백 루프. 실험(A/B)은 model_version으로 버전을 서빙한다.

1. 레이어별 기술

레이어	무엇을 하나	기술
서빙 (실시간)	score() 추론 + 슬레이트 + 온라인 SGD	Node/Express + TypeScript (기존 server/engine)
피처 스토어	x_i / θ_u / 이벤트 로그	Postgres(Supabase) + Drizzle + pgvector
오프라인 (배치)	임베딩 / IPS 학습 / survival / off-policy	Python (numpy/scikit-learn, lifelines, sentence-transformers)
실험	배정 / 판정	assignVariant + Tier 4 readout (구축 완료)

2. 모듈형 서브스코어러

스코어러는 하나의 거대 함수가 아니라, 교체 가능한 서브스코어러의 합이다. 각 항을 휴리스틱 -> 학습형으로 하나씩 교체한다. 이 구조가 곧 B2B 이식성(외부가 자기 피쳐만 꽃음).

```
score(c | user, ctx) =  
  w1*취향 + w2*맥락 + w3*평판 + w4*새로움 + w5*그룹  
  - w6*노출피로 +/- w7*재소비(satiation) + eps(탐색)
```

- 인터페이스: SubScorer.score(candidate, ctx, userState) -> number. 파이프라인이 가중합.
- v0은 평판/맥락만 휴리스틱. 비어있는 취향/노출피로/satiation 항을 로그로 학습해 채운다.
- 가중치 w1..w7은 맥락별로 동적(점심 vs 디저트, 자국 vs 여행). 최종적으로 학습 대상.

3. 데이터 흐름 / 피드백 루프

- 1) 앱 -> /api/recommend: 맥락 스냅샷과 함께 추천 요청.
- 2) 서버: FeatureProvider가 스토어에서 θ_u/x_i 를 읽어 스코어 -> 슬레이트(K개 + propensity).
- 3) 노출/스와이프가 rec_events에 로깅(propensity 승계). 온라인 학습기는 스와이프마다 θ_u 를 즉시 SGD 갱신.
- 4) 오프라인(야간): rec_events를 ETL -> IPS 배치 학습/임베딩/survival -> 스토어 갱신.
- 5) 서버가 갱신된 피처를 다시 읽는다. 루프가 닫힌다.

4. 학습 메커니즘 (v1 + off-policy)

취향 벡터 θ_u

- 라벨: 모든 SWIPE = LIKE(1)/NOPE(0), 아이템 x_i 에 대한 암묵적 응답.
- 모델: $P(\text{like}|u,i) = \text{sigmoid}(\theta_u \cdot x_i + b)$ — 아이템 피쳐 위 유저별 로지스틱.
- 온라인: $\theta_u \leftarrow \theta_u + \eta(y - p)x_i$ (스와이프 즉시 반영). 오프라인: IPS 가중 $1/\text{propensity}$ 로 재적합.
- 콜드스타트: 신규 유저 θ_u = 모집단 평균(계층 베이스 shrink) -> 스와이프 쌓이며 개인화.

노출 피로 / off-policy

- 노출 피로: $-w_6 \cdot g$ (누적 노출). g 는 Tier 2 노출피로 패널의 실제 감소곡선에서 적합.
- off-policy: v0의 편향된 슬레이트를 $1/\text{propensity}$ 로 보정(IPS). 출시 전 IPS/DR로 $V(\pi_{v1})$ 추정 -> v0보다 나으면 Tier 4 A/B로 온라인 확정.

5. 로드맵 — 단계 / 학습신호 / 검증패널

단계	학습 신호	모델	검증 패널
v0.5 아이템 피쳐	메뉴/리뷰/태그(오프라인)	구조적+맛 프로파일+임베딩 $\rightarrow x_i$	(기반)
v1 취향+노출피로	스와이프 + 노출횟수	θ_u 로지스틱 + 피로 패널티	Tier 2 분별력/노출피로
v2 satiation+연쇄	소비 timestamp(검열)	생존분석 hazard + occasion 시퀀스	Tier 2 satiation
v3 밴딩+그룹	propensity + 보상	Thompson Sampling + least-misery	Tier 2 탐색/공정성
v4 전이+명시신호	새 도시 스와이프 + 저장/수정	city-invariant 부분공간 매핑	Tier 2 여행전이

6. 재사용 vs 신규 / 규모 확장

- 재사용: scorer.ts(서브스코어러로 리팩터), events.ts(피드백 수집), context.ts, Tier 2/4 패널(검증 하니스), propensity 로깅(off-policy 연료).
- 신규: restaurant_features/user_taste 테이블(+pgvector), 온라인 SGD 학습기(TS), Python 배치 잡 1개.
- 확장: neural로 가면 Python 추론 서비스(FastAPI) 분리. 카탈로그 거대화 시 pgvector $\rightarrow$ 전용 벡터DB. 24개 식당 규모에선 인라인 TS + Postgres가 정답(과설계 금물).

7. 첫 스프린트

- 1) v0.5: restaurant_features 테이블 + 맛 프로파일 추출(구조적 먼저, 임베딩 후속).
- 2) v1: scoreCandidate에 $w_1 * \text{sigmoid}(\theta_u \text{ dot } x_i)$ 추가 + 온라인 SGD + IPS 야간 배치.
- 3) 검증: Tier 2 분별력 gap 상승, Tier 4 A/B에서 v1 arm이 random/v0를 이기는지.